Complete Project Explanation

Steps 1–5: Secure Automotive Event Logging using ARM TrustZone and OP-TEE

Raspberry Pi 3 Model B / B+ · ARM64/AArch64

Field	Details
Student	Aslam
Prepared	2 September 2026
Completed	Steps 1, 2, 3 and 4
In progress	Step 5 — Buildroot root filesystem
Project tree	/home/aslam/Minor_Project/optee_full

Purpose of this document: These notes explain what was done, why each step was necessary, how the steps connect, what each output means, and which claims can or cannot yet be made.

1. The Main Project Idea

The final objective is to receive automotive CAN packets and validate them in ARM TrustZone's Secure World before releasing approved information to ordinary Linux. Security-sensitive keys, validation rules, critical-event decisions, and tamper-evident logs are intended to remain under OP-TEE control.

```
Sender ECU → CAN bus → CAN controller/transceiver
→ secure-designated GPIO interrupt → secure SPI/CAN receive path
→ Secure World validation
■■■ critical packet → secure handling + secure log
■■■ normal accepted packet → controlled result to Linux
```

The planned checks

- CAN-ID policy: decide whether an identifier is allowed and whether it is critical.
- HMAC authentication: verify that the packet came from an entity that knows the secret key.
- Counter and freshness checks: reject old or delayed data.
- Replay detection: prevent reuse of a previously valid packet.
- Flood/rate detection: detect excessive packet frequency.
- Classification and logging: keep critical events in Secure World and create a hash-chained tamper-evident log.

Research-platform limitation

Raspberry Pi 3 is useful for demonstrating TrustZone concepts, Secure/Normal World transitions, Trusted Applications, client communication, and secure validation logic. It must not be described as production-grade automotive isolation because the RPi3 OP-TEE port lacks production-grade secure DRAM/peripheral isolation and a complete production secure-boot chain.

2. TrustZone and the Two Worlds

Secure World	Normal World
OP-TEE OS	U-Boot and Linux
Trusted Applications	Client applications
Protected keys and validation	Networking, filesystems, shell and ordinary services
Security decisions	Receives only approved results

```
Normal application → libteec → Linux OP-TEE driver
→ Secure Monitor Call (SMC) → OP-TEE OS → Trusted Application
```

A compile-time result only proves that source code produced binaries. Runtime proof requires the board to boot the binaries and demonstrate real communication across this entire path.

3. How the Boot Components Connect

```
Raspberry Pi Boot ROM / VideoCore firmware
↓
armstub8.bin
■■■ TF-A BL1: early secure firmware
■■■ FIP (Firmware Image Package)
■■■ BL32: OP-TEE OS — Secure World
■■■ BL33: U-Boot — Normal World bootloader
↓
U-Boot loads Linux Image + correct Device Tree
↓
Linux mounts Buildroot root filesystem
↓
tee-supplicant + libteec + xtest + applications + TA files
```

Component	Why it is required
OP-TEE OS	Provides the Secure World operating system and executes Trusted Applications.
TF-A	Provides early secure firmware and the Secure Monitor used for world transitions.
U-Boot	Loads the Linux kernel, DTB and boot parameters in Normal World.
Linux	Runs ordinary applications, drivers, storage and the OP-TEE kernel interface.
Device Tree	Describes the Raspberry Pi hardware to Linux.
Buildroot filesystem	Provides commands, libraries, services, clients, tests and TA files after Linux starts.

4. Step 1 — Build OP-TEE OS

Status: complete. In this step, OP-TEE OS was cross-compiled for the Raspberry Pi 3 AArch64 target. The build ran on an x86-64 laptop but generated ARM64 machine code for the Raspberry Pi.

Why Step 1 was necessary

The secure part of the final system needs an operating environment in which Trusted Applications can execute. OP-TEE OS supplies secure sessions, memory management, cryptographic APIs, secure storage support, and controlled communication with Linux.

What was generated

Output	Meaning
tee.bin	Combined OP-TEE binary; verified size 532,228 bytes.
tee.elf	ELF image containing symbols useful for debugging.
tee-header_v2.bin	Header supplied to TF-A for the split OP-TEE image.
tee-pager_v2.bin	Main pager/secure payload content.
tee-pageable_v2.bin	Pageable portion; zero bytes can be valid for this build arrangement.

What Step 1 proves

It proves that OP-TEE compiled successfully for the selected RPi3 platform and produced the BL32 payload files required by TF-A.

It does **not** prove that OP-TEE booted on the physical Raspberry Pi, that a TA ran, or that Linux communicated with OP-TEE.

Presentation answer

“I cross-compiled OP-TEE OS for Raspberry Pi 3 using AArch64. OP-TEE will run in TrustZone Secure World and execute security-sensitive Trusted Applications. The build produced tee.bin and the split images required by TF-A. This proves compilation, not physical-board execution.”

5. Step 2 — Build U-Boot and Trusted Firmware-A

Status: complete. This step created the secure monitor/firmware layer and the Normal World bootloader, then packaged them with the Step 1 OP-TEE output.

Trusted Firmware-A

TF-A initializes the ARM processor in the secure boot stage, provides the runtime Secure Monitor, and controls transitions between Secure and Normal Worlds. OP-TEE is configured as BL32 and U-Boot as BL33.

U-Boot

U-Boot is the Normal World bootloader. Its later job is to load the ARM64 Linux `Image`, select the correct Raspberry Pi DTB, provide boot arguments, and start Linux.

```
fip.bin
■■■ BL32 = OP-TEE OS
■■■ BL33 = U-Boot

armstub8.bin = TF-A BL1 padded to 0x20000 + fip.bin
```

Verified output	Size	Purpose
u-boot.bin	540,496 bytes	Normal World bootloader
fip.bin	1,147,912 bytes	Package containing later firmware payloads
armstub8.bin	1,278,984 bytes	RPi3 early-boot file containing BL1 and the FIP

Important packaging point

For this TF-A RPi3 build, `fip.bin` is already appended inside `armstub8.bin`. Therefore, the FIP should not be copied separately to the boot partition for this selected build flow.

What Step 2 proves

It proves that U-Boot and TF-A compiled and that OP-TEE and U-Boot could be packaged as BL32 and BL33. It still does not prove that the combined chain boots on the Raspberry Pi.

6. Step 3 — Build Linux and Existing Device Trees

Status: complete. The ARM64 Linux kernel source was compiled, and the standard existing Raspberry Pi Device Tree sources in the same Linux tree were compiled into binary DTB files.

Why Linux is required

OP-TEE is not a replacement for the complete Linux environment. Linux runs ordinary applications, networking, storage, processes, device drivers, and the Normal World side of OP-TEE communication. In the final design Linux should receive only approved information after Secure World validation.

What configuration changed

The kernel configuration was prepared for ARM64/Raspberry Pi 3 and the following support was verified:

```
CONFIG_TEE=y
CONFIG_OPTEE=y
```

Option	Meaning
CONFIG_TEE=y	Build Linux's generic Trusted Execution Environment subsystem directly into the kernel.
CONFIG_OPTEE=y	Build the OP-TEE-specific Linux driver directly into the kernel.

Linux configuration versus Device Tree

Linux configuration	Device Tree
Selects software features and drivers compiled into Linux.	Describes the board's physical hardware to Linux.
We verified TEE and OP-TEE software support.	We did not add new hardware or create a new Device Tree.
Produces the kernel Image and modules.	Existing .dts/.dtsi sources are compiled into .dtb files.

Why compile DTBs when no new hardware was attached?

No custom hardware was added in Step 3 and no custom CAN node was created. The standard Device Trees were compiled because a bootable ARM system needs a DTB and because using the DTB produced from the same controlled Linux source revision keeps the kernel/driver/hardware description set reproducible. This was normal compilation of existing source, not a hardware modification.

```
Existing bcm2710-rpi-3-*.dts/.dtsi
→ Device Tree Compiler (DTC)
→ bcm2710-rpi-3-b.dtb and bcm2710-rpi-3-b-plus.dtb
```

Outputs and verification

Artifact	Size	Reason
Image	22,993,408 bytes	Uncompressed ARM64 Linux kernel loaded by U-Boot
linux.config	215,638 bytes	Exact reproducible kernel configuration
bcm2710-rpi-3-b.dtb	33,616 bytes	Hardware description for RPi3 Model B
bcm2710-rpi-3-b-plus.dtb	34,235 bytes	Hardware description for RPi3 Model B+

The files were collected under `Step3_Output/`, then their sizes and SHA-256 checksums were recorded.

What Step 3 does not prove

Enabling the OP-TEE driver only gives Linux the ability to communicate if a compatible Secure World is actually running. Step 3 cannot prove that `/dev/tee0` exists, that `tee-supplicant` runs, or that a client can open a TA session.

Presentation answer

“I compiled the ARM64 Linux kernel after enabling the generic TEE subsystem and OP-TEE driver. I also compiled the existing standard Raspberry Pi 3 Device Tree sources. No new hardware was added and the Device Tree was not customized. I verified and preserved the kernel Image, configuration, and both board DTBs. Runtime communication still requires the root filesystem and physical boot.”

7. Step 4 — Verification and Organization

Status: complete. Step 4 did not add a new runtime component. It created a reliable and reproducible foundation before attempting the complete filesystem and image build.

What was done

- Selected the complete laptop build tree: /home/aslam/Minor_Project/optee_full.
- Confirmed the real complete Pi tree: /home/aslam/Minor/optee_project. The Pi's /home/aslam/Minor_Project/optee_full directory was empty and excluded.
- Verified the presence of build, Buildroot, firmware, Linux, OP-TEE OS/client/test/examples, TF-A, and U-Boot.
- Compared component Git revisions between laptop and Pi; they matched.
- Backed up successful Step 1–3 artifacts under Step4_Verification/verified_artifacts.
- Calculated SHA-256 checksums to detect later replacement or corruption.
- Inspected build/rpi3.mk and confirmed official targets instead of guessing commands.

Supported target	Purpose
tf-a	Build/package OP-TEE and U-Boot through TF-A
buildroot	Build the root filesystem after update_rootfs
update_rootfs	Install matching kernel modules into Buildroot target
update_bootfs	Assemble firmware, kernel, DTBs, armstub8.bin and U-Boot environment
sdcard-image	Generate out/rpi3-sdcard.img
all	Run the complete dependency and image chain

Consistency issue found

The laptop `tee.bin` had been rebuilt after the existing `armstub8.bin` was packaged. Consequently, the current loose OP-TEE binary may not be the OP-TEE payload inside that older armstub. The supported dependency chain must be rerun before final packaging so OP-TEE, U-Boot, and TF-A correspond.

Why Step 4 matters

It prevents using an empty tree, mixing source revisions, losing successful binaries, packaging mismatched payloads, or using an unsupported command. Step 4 is quality control and reproducibility work, not runtime verification.

8. Step 5 — Buildroot Root Filesystem and OP-TEE User Space

Status: in progress and currently stopped at a toolchain-path error. Step 5 creates the Linux environment that begins running after the kernel boots.

Buildroot versus the root filesystem

Buildroot	Root filesystem
A build system that runs on the laptop.	The directory/image mounted by Linux as /.
Cross-compiles packages for ARM64.	Contains commands, libraries, configuration and services.
Creates images such as rootfs.ext4.	Runs on the Raspberry Pi after boot.

```
Laptop → Buildroot
■■■ cross-compile ARM64 programs
■■■ install matching kernel modules
■■■ install OP-TEE client/test/example packages
■■■ generate rootfs.ext4
↓
Raspberry Pi root partition
```

Why a kernel alone is not enough

The Linux `Image` supplies the kernel, but it does not supply a shell, login, shared libraries, daemon processes, client applications, configuration, or TA files. The root filesystem supplies directories such as `/bin`, `/sbin`, `/lib`, `/usr`, `/etc`, and `/lib/optee_armtz`.

What has actually been done

- Started the official rpi3.mk buildroot target with four parallel jobs.
- Rechecked OP-TEE dependencies and existing build outputs.
- Continued the required Linux kernel and module build.
- Generated approximately 1,713 module-related files for installation in the target filesystem.
- Entered the Buildroot package configuration and compilation phase.
- Saved the full output in step5-buildroot-2026-09-01.log.
- Identified the current blocking cross-compiler name/path mismatch.

Current failure

Buildroot stopped because it attempted to execute `toolchains/aarch64/bin/aarch64-linux-gnu-gcc`, while the available downloaded toolchain exposes the `aarch64-none-linux-gnu-gcc` prefix. The toolchain configuration/path must be corrected, then the incremental Buildroot target can resume.

9. OP-TEE User-Space Components in Step 5

9.1 tee-supplicant

`tee-supplicant` is a Normal World background daemon. OP-TEE uses it when Secure World needs controlled assistance from Linux, including loading TA files, backing secure-storage objects with normal-world files, RPC services, or plugins.

```
Trusted Application / OP-TEE OS
→ RPC request through Linux OP-TEE driver
→ tee-supplicant
→ normal-world TA file or storage file
→ response returned to Secure World
```

tee-supplicant is not trusted merely because it assists OP-TEE. It runs in Normal World. OP-TEE must authenticate/protect security-sensitive content and should not expose secret plaintext to the daemon.

9.2 libteec

`libteec` implements the GlobalPlatform-style TEE Client API for Normal World programs. It converts convenient application calls into requests for the Linux OP-TEE kernel driver.

```
TEEC_InitializeContext()
TEEC_OpenSession()
TEEC_InvokeCommand()
TEEC_CloseSession()
TEEC_FinalizeContext()
```

For the future CAN project, a Linux client may invoke an approved command through libteec and receive only the result that the secure TA is allowed to release.

9.3 xtest

`xtest` is the OP-TEE test suite. It exercises client sessions, parameter transfer, shared memory, cryptography, secure storage, error handling, cancellation, timing, random functions, and test TAs. It provides stronger runtime evidence than the existence of tee.bin.

```
xtest → libteec → Linux OP-TEE driver → SMC
→ OP-TEE OS → test TA → result returned to xtest
```

9.4 Host applications

A host application is an ordinary Linux program that opens a session with a particular Trusted Application, invokes commands, sends permitted inputs, and receives approved outputs. Examples include OP-TEE hello-world and secure-storage clients.

9.5 Trusted Applications (.ta files)

A TA contains security-sensitive code executed by OP-TEE. It is normally identified by a UUID and stored as `<UUID>.ta` in the root filesystem. The host and TA must agree on the UUID, command IDs, parameter types, and shared data structures.

```
Host: TEEC_OpenSession(TA_CAN_UUID)
Host: TEEC_InvokeCommand(CMD_VALIDATE_PACKET)
TA: same UUID + handler for CMD_VALIDATE_PACKET
```

9.6 Matching kernel modules

Kernel modules are separately loadable drivers/features installed under `/lib/modules/<kernel-version>`. They must come from the same kernel build as the Linux Image. Modules from a different kernel version can fail version checks or behave incompatibly.

10. When Can Linux–OP-TEE Communication Be Claimed?

Not after Step 3 and not merely after compiling the root filesystem. The complete boot chain must run on the board.

Evidence	What it proves
tee.bin exists	OP-TEE compiled
CONFIG_OPTEE=y	Linux contains the OP-TEE driver
rootfs contains libteec/tee-supplicant	User-space components were packaged
OP-TEE messages in boot log	Secure World started
/dev/tee0 and /dev/teepriv0	Linux detected a TEE interface
tee-supplicant running	Normal World RPC daemon is active
xtest/example succeeds	End-to-end Linux/client/driver/SMC/OP-TEE/TA communication works

Runtime commands to collect evidence

```
dmesg | grep -i -E 'op-tee|optee|tee'
ls -l /dev/tee*
ps | grep tee-supplicant
xtest
optee_example_hello_world
```

The currently running Raspberry Pi OS was checked earlier and had no /dev/tee* devices, no OP-TEE boot messages, and no armstub8.bin in the active boot set. Therefore, physical OP-TEE runtime communication has not yet been demonstrated.

11. Full Step-to-Step Connection

Step	Main action	Result	Boundary
1	Compile OP-TEE OS	Secure World binary	Compilation only
2	Build TF-A + U-Boot and package BL32/BL33	armstub8.bin/FIP/U-Boot	Packaging only
3	Compile Linux + existing RPi3 DTBs	Image, config and DTBs	Driver support only
4	Verify tree, revisions, artifacts, checksums and targets	Controlled reproducible base	No new runtime component
5	Build modules + Buildroot OP-TEE user space	Expected rootfs.ext4	Currently incomplete

Step 1 gives Secure World.
Step 2 gives the secure monitor and bootloader packaging.
Step 3 gives the Normal World kernel and hardware description.
Step 4 ensures the inputs are correct and reproducible.
Step 5 gives Linux user space and the client/TA runtime files.
Later steps assemble the boot image, boot the board, verify OP-TEE, and implement secure CAN.

12. Remaining Work After the Current Point

1. Correct the AArch64 Buildroot toolchain prefix/path and resume the incremental build.
2. Verify rootfs.ext4, tee-supplicant, libteec, xtest, host examples, TA files and shared-library dependencies.
3. Repackage OP-TEE, U-Boot and TF-A consistently through the supported dependency chain.
4. Run update_bootfs to assemble Raspberry Pi firmware, config, kernel, DTB, armstub8.bin and U-Boot environment.
5. Generate rpi3-sdcard.img and record its checksum.
6. Identify the physical SD-card block device safely before any destructive write.
7. Boot the Raspberry Pi with serial capture and verify Secure World plus Linux.
8. Run xtest and example client/TA communication.
9. Attach/configure the external CAN controller only in the later hardware phase.
10. Implement secure SPI/interrupt ownership, CAN validation policies and hash-chained logging.
11. Perform functional, replay, freshness, HMAC and flood tests.

13. Common Questions and Correct Answers

Did we add hardware in Step 3?

No. We compiled the existing standard RPi3 Device Tree sources. Custom CAN hardware and secure ownership are later work.

Why rebuild a Device Tree if the Pi already has one?

To preserve a controlled kernel-and-DTB set from the same selected source revision. We did not claim that the existing Pi DTB was absent or that hardware had changed.

Did CONFIG_OPTEE=y prove communication?

No. It only built the driver capability into Linux. Runtime detection and an xtest/example success are required.

Did Step 4 build anything new?

No new runtime component. It verified source completeness, revisions, outputs, integrity and supported build targets.

Is Step 5 complete?

No. Kernel/module work progressed and Buildroot started, but the final OP-TEE root filesystem was not generated because the cross-compiler path/prefix did not match.

14. Complete Oral Explanation

“My project aims to validate automotive CAN messages inside ARM TrustZone before releasing approved information to Linux. In Step 1, I cross-compiled OP-TEE OS for Raspberry Pi 3, producing the Secure World payload. In Step 2, I built Trusted Firmware-A and U-Boot. TF-A provides the Secure Monitor, loads OP-TEE as BL32, and packages U-Boot as BL33. For Raspberry Pi 3, the padded TF-A BL1 and FIP are combined into armstub8.bin.

“In Step 3, I compiled a 64-bit Linux kernel and enabled CONFIG_TEE and CONFIG_OPTEE so the Normal World kernel contains the OP-TEE interface. I also compiled the existing standard Raspberry Pi 3 Device Tree sources into DTBs. No new hardware was attached and the Device Tree was not customized. I preserved the Linux Image, configuration and DTBs, but this still did not prove runtime communication.

“In Step 4, I selected the complete authoritative project tree, compared laptop and Pi source revisions, backed up successful outputs, recorded checksums, and verified the official Raspberry Pi build targets. This provided a consistent base and revealed that some loose OP-TEE and packaged TF-A artifacts had different build times.

“In Step 5, I started Buildroot to create the Linux root filesystem. The root filesystem must contain matching kernel modules, tee-supplicant, libteec, xtest, host applications and Trusted Applications. tee-supplicant provides normal-world RPC services, libteec is the client API used by applications, xtest checks the full communication path, and TA files execute security-sensitive commands in OP-TEE. Kernel modules were built and Buildroot started, but the final rootfs is not complete because the expected AArch64 compiler prefix does not match the available toolchain. After fixing that, I will resume Buildroot, assemble the boot image, boot the board, and verify end-to-end communication before implementing secure CAN.”

15. Final Status

Area	Status
OP-TEE OS compilation	Complete
TF-A and U-Boot compilation/packaging	Complete
ARM64 Linux and standard RPi3 DTBs	Complete
Source/artifact verification and backup	Complete
Buildroot root filesystem	In progress — toolchain mismatch
Boot partition and SD-card image	Not complete
Physical OP-TEE boot and xtest	Not complete
Secure CAN implementation	Not started

The strongest correct claim today is: **the secure/normal-world build components through Step 4 are compiled and verified as artifacts; Step 5 has progressed into Buildroot but the usable runtime filesystem and physical OP-TEE communication are not yet complete.**

Evidence locations: Step3_Output/, Step4_Verification/, CURRENT_STATUS_AND_NEXT_STEPS.md, RPI3_SECURE_CAN_FULL_ROADMAP.md, and step5-buildroot-2026-09-01.log.